Co Language Specification

0. Introduction

Co is a garbage-collected, dynamically-typed, imperative programming language with first-class functions, cooperative coroutines, and channel-based concurrency.

This document specifies the complete semantics of Co. The language is designed to be minimal yet expressive, with a straightforward implementation. It draws inspiration from languages like JavaScript, and Go, while maintaining a distinct identity focused on simplicity and coroutine-based concurrency.

The specification covers ten major areas: lexical structure (tokens, comments, and identifiers), the type system (primitives, truthiness, and equality), expressions (literals, operators, and function calls), statements (variable declarations, assignments, control flow, and function definitions), scoping rules (lexical scoping with run-time enforcement), concurrency primitives (coroutines, channels, spawning, and yielding), builtin functions (print, channels, and timing), error handling (run-time errors and termination), program structure (top-level statements and execution model), and a formal grammar.

The language is interpreted and executes programs composed of statements at the top level. Control flow is provided through `if` conditionals and `while` loops. Functions are first-class values supporting closures. Concurrency is cooperative—coroutines yield explicitly via the `yield` keyword, and communication occurs through channels using send (`->`) and receive (`<-`) operators.

1. Lexical Structure

1.1 Comments

- **Line comments**: `// ...` until end of line.
- **Block comments**: `/* ... */`, may span multiple lines.

1.2 Whitespace

All whitespace (spaces, tabs, newlines) is insignificant except as a token separator.

1.3 Identifiers

An identifier starts with a letter (`a-z`, `A-Z`) followed by zero or more alphanumeric characters (`a-z`, `A-Z`, `0-9`). Identifiers are case-sensitive.

1.4 Reserved Words

The following words are reserved and cannot be used as identifiers:

```
null  true  false  var  if  while  function  return  yield  spawn
```

1.5 Literals

Type	Syntax	Examples
Null	`null`	`null`
Boolean	`true`, `false`	`true`, `false`
Integer	Optional sign followed by digits	`42`, `-1`, `0`
String	Double-quoted character sequence	`"hello"`, `""`

String literals support standard character escape sequences (e.g. `\"`, `\\`, `\n`).

1.6 Operators and Punctuation

```
+  -  *  /  ==  !=  <  >  =  <-  ->  (  )  {  }  ;  ,
```

2. Types and Values

Co is dynamically and strongly typed. Values have the following types at run-time:

Type	Description
Null	The singleton `null` value.
Boolean	`true` or `false`.
Number	Arbitrary-precision integer.
String	Sequence of characters.
Function	Named function, lambda, or builtin function.
Channel	Communication channel between coroutines.

2.1 Truthiness

Values are interpreted as booleans in conditional contexts (`if`, `while`):

- **Falsy**: `null`, `false`
- **Truthy**: Everything else, including `0`, `""` (empty string), functions, and channels.

2.2 Equality

- `null == null` is `true`.
- Booleans, numbers, and strings compare by value.
- Functions and channels do not support meaningful equality (always `false` when compared to each other or across instances).
- `!=` is the logical negation of `==`.

2.3 String Representation

Each value type has a string representation used by `print` and string coercion:

Value	String Representation
`null`	`null`
`true`	`true`
`false`	`false`
Number `n`	Decimal digits (e.g. `42`, `-1`)
String `s`	The string itself
Function	`function <name>` (lambdas use `function <lambda>`)
Channel	`Channel`

2.4 String Coercion

When the `+` operator is used with at least one string operand, the non-string operand is coerced to its string representation (see §2.3):

```
"count: " + 42       // "count: 42"
true + " value"      // "true value"
```

3. Expressions

Expressions are evaluated to produce values.

3.1 Literal Expressions

```
null
true
false
42
-1
"hello world"
```

3.2 Variable Reference

`identifier`

Evaluates to the current value bound to the identifier. It is an error to reference a variable that has not been defined in the current scope or an enclosing scope.

3.3 Binary Operators

`expr op expr`

Binary operators evaluate the left operand first, then the right operand.

Arithmetic Operators (numbers only)

Operator	Operation	Result Type
`+`	Addition (or string concat/coercion)	Number or String
`-`	Subtraction	Number
`*`	Multiplication	Number
`/`	Integer division (truncated toward negative infinity)	Number

Arithmetic operators (except `+`) require both operands to be numbers. The `+` operator is overloaded:

- If both operands are numbers: numeric addition.
- If both operands are strings: string concatenation.
- If one operand is a string: the other is coerced to a string, then concatenated.
- Otherwise: run-time error.

Comparison Operators (numbers only)

Operator	Operation	Result Type
`<`	Less than	Boolean
`>`	Greater than	Boolean

Both operands must be numbers.

Equality Operators (any type)

Operator	Operation	Result Type
`==`	Equal	Boolean
`!=`	Not equal	Boolean

Operator Precedence (highest to lowest)

1. Function call `expr(args)` (left-to-right chaining)
2. `<-` (receive, prefix)
3. `*`, `/`
4. `+`, `-`
5. `<`, `>`
6. `==`, `!=`

All binary operators are left-associative. There is no unary minus operator; negative numbers are parsed as signed integer literals.

3.4 Function Call

```
expr(arg1, arg2, ...)
```

The callee expression is evaluated first, then arguments are evaluated left-to-right. The callee must evaluate to a function (named, lambda, or builtin). The number of arguments must exactly match the function's arity. Calls can be chained: `f(1)(2)` calls the result of `f(1)` with argument `2`.

3.5 Lambda Expression

```
function(param1, param2, ...) { body }
```

Creates an anonymous function value. The lambda captures the environment at the point of its creation, forming a closure. Lambdas can be immediately invoked (IIFE):

```
var result = function(x, y) { return x + y; }(1, 2);
```

3.6 Receive Expression

```
<- expr
```

Receives a value from a channel. The expression must evaluate to a channel. This is a prefix unary operator. If no value is available, the current coroutine blocks until a sender provides one.

4. Statements

Statements are executed for their effects and do not produce values (except `return`). Most statements are terminated by `;`. Block-based statements (`if`, `while`, `function`) are not.

4.1 Expression Statement

```
expr;
```

Evaluates the expression and discards the result.

4.2 Variable Declaration

`var identifier = expr;`

Declares a new variable and initializes it with the value of `expr`. A variable **must** be initialized at declaration. It is an error to:

- Declare a variable with a name that is already defined in the current scope (see §5).
- Reference the variable being declared within its own initializer expression.

4.3 Assignment

`identifier = expr;`

Assigns a new value to an existing variable. The variable must have been previously declared (via `var`, `function`, as a parameter, or as a builtin). Assignments can modify variables in enclosing scopes (closures). Function parameters and builtins may be reassigned. Reassgining function arguments in function bodies does not change the value of arguments at the callers of the functions.

4.4 If Statement

`if (expr) { body }`

Evaluates `expr`, and if it is truthy (see §2.1), executes the body. There is no `else` clause. The `if` block does **not** introduce a new scope (see §5). Since scoping is enforced at run-time, variables declared inside a non-taken `if` branch will not exist:

```
if (false) { var x = 1; }
print(x);  // ERROR: Unknown variable: x
```

4.5 While Statement

`while (expr) { body }`

Repeatedly evaluates `expr` and, while truthy, executes the body. The `while` block does **not** introduce a new scope (see §5). However, variables declared *inside* a `while` body may be redefined across iterations only if they were first defined inside the `while` (not outside).

4.6 Function Declaration

`function name(param1, param2, ...) { body }`

Declares a named function and binds it to `name` in the current scope. The function captures the environment at the point of its definition, forming a closure. It is an error to:

- Use the same name for the function and one of its parameters.
- Have duplicate parameter names.
- Redefine a name that is already bound in the enclosing scope.

The function name is available for recursive calls within its own body via an implicit self-reference binding.

4.7 Return Statement

```
return;
return expr;
```

Returns a value from the current function. If no expression is given, `null` is returned. A `return` without an enclosing function (i.e., at global scope, or inside `if`/`while` at global scope) is an error. Functions that reach the end of their body without a `return` implicitly return `null`.

4.8 Yield Statement

```
yield;
```

Cooperatively yields execution to the next coroutine in the scheduler queue. The current coroutine is re-enqueued and is resumed later. `yield` is valid at global scope, inside functions, and inside lambdas.

4.9 Spawn Statement

```
spawn expr;
```

Schedules `expr` for execution as a new coroutine. Typically used with a function call:

```
spawn work();
spawn function() { ... }();
```

The spawned expression is evaluated in a new coroutine. The coroutine captures the current environment at the point of the `spawn` statement; since variables are mutable references, the spawned coroutine shares mutable state with the spawning coroutine. The `spawn` statement returns immediately; the spawned coroutine is scheduled to run later.

4.10 Send Statement

```
value_expr -> channel_expr;
```

Sends the value of `value_expr` to the channel that `channel_expr` evaluates to. The channel expression (right side) is evaluated first, then the value expression (left side). The channel expression must evaluate to a channel. Behavior depends on the channel type:

- If a receiver is already waiting: the value is delivered directly and the receiver is scheduled.
- If the channel is buffered and not full: the value is placed in the buffer.
- If the channel is unbuffered or the buffer is full: the sender blocks until a receiver is available (up to a maximum send queue size of 4).

5. Scoping Rules

Co has a strict scoping rules enforced at run-time.

5.1 Scope Types

There are two fundamental scope types:

1. **Global scope**: The top-level of a program.
2. **Function scope**: Created by function declarations and lambda expressions.

`if` and `while` blocks do NOT create new scopes. Variables declared inside `if` or `while` bodies are visible in the enclosing scope after the block.

```
if (true) { var x = 1; }
print(x);  // OK: x is visible here
```

5.2 Variable Resolution

Variables are resolved lexically (statically). A variable reference is valid only if the variable has been defined *before* the point of use in the source text. There is **no hoisting**—a variable cannot be used before its declaration, even within the same scope.

```
print(x);     // ERROR: Unknown variable: x
var x = 1;
```

This applies within function bodies as well:

```
function f() {
  var a = b;  // ERROR: Unknown variable: b
  var b = 1;
}
```

5.3 Redefinition Rules

A variable **cannot** be redefined in the same scope or in an enclosed scope that sees the original binding. The following cause errors:

- Redefining a `var` in the same scope.
- Redefining a `var` inside a function/lambda that captures the outer scope.
- Redefining a builtin with `var` or `function`.
- Redefining a function name with `var` (or vice versa).
- Redefining a function parameter with `var` inside the function body.
- Defining a function's own name as a `var` inside its body.
- Defining a variable in an `if`/`while` block that conflicts with the enclosing scope.
- Defining the same variable in two separate `if` blocks at the same scope level.

```
var x = 1;
var x = 2;     // ERROR: Variable already defined: x

var y = 1;
if (true) { var y = 2; }  // ERROR: Variable already defined: y

function f() { var f = 2; }  // ERROR: Variable already defined: f
```

Exception: `while`-local variables

Variables defined for the first time inside a `while` body (that did not exist before the while) are allowed and treated as belonging to the enclosing scope. These `while`-local variables may be re-declared across loop iterations without error (since each iteration re-executes the `var` statement). However, redefining a variable from the outer scope inside a `while` is still an error.

5.4 Shadowing

Shadowing is permitted **only** through function/lambda parameters:

- A function parameter may shadow an outer variable, function, or builtin.
- A lambda parameter may shadow an outer variable, function, or builtin.
- Nested lambdas may each shadow the same name through their own parameters.

```
var x = 1;
function f(x) { return x; }    // OK: parameter shadows outer var
var g = function(print) { };    // OK: parameter shadows builtin
```

5.5 Function Scope Capture

Functions and lambdas capture the environment's variable bindings (references) at the point of their definition. They can mutate the captured variables, and can

observe mutations made to captured variables after their definition, but they cannot reference variables defined *after* their definition.

```
function f() { return laterVar; }  // ERROR: Unknown variable: laterVar
var laterVar = 1;
```

5.6 Self-Reference

- **Named functions** can refer to themselves by name within their body, enabling recursion.
- **Lambdas** cannot refer to the variable they are being assigned to within their body. The lambda captures the environment before the variable is defined; calling the lambda later fails with an error.
- **`var` declarations** cannot reference themselves in their initializer, since the initializer is evaluated before the variable is added to the environment.

```
function fib(n) { return fib(n-1) + fib(n-2); }  // OK

var x = function() { return x; };
x();  // ERROR: Unknown variable: x

var y = y + 1;  // ERROR: Unknown variable: y
```

5.7 Nested Function and Variable Visibility

A function or variable declared inside a function is not visible outside the outer function:

```
function outer() {
  function inner() { return 1; }
  var x = 1;
  return inner();  // OK
}
inner();  // ERROR: Unknown variable: inner
print(x); // ERROR: Unknown variable: x
```

5.8 Return Scope Rules

`return` is only valid inside a function scope. A `return` inside an `if` or `while` at global scope is an error:

```
return 42;                   // ERROR: Cannot return from global scope
if (true) { return 1; }      // ERROR: Cannot return from global scope
while (true) { return 1; }   // ERROR: Cannot return from global scope
```

But inside a function, `return` in nested `if`/`while` is fine:

```
function f() {
  if (true) { return 42; }  // OK
```

```
}
```

6. Concurrency

Co supports cooperative concurrency through coroutines and channels.

6.1 Coroutines

Coroutines are cooperatively scheduled. A coroutine runs until it explicitly yields control via:

- `yield`: explicitly yields to the scheduler.
- A blocking channel operation (`send` to a full/unbuffered channel, `receive` from an empty channel).
- `sleep(ms)`: suspends for at least `ms` milliseconds.

The main program runs as the initial coroutine. After the main body finishes, the runtime awaits termination of all remaining coroutines.

6.2 Channels

Channels are untyped communication primitives for passing values between coroutines.

Unbuffered Channels

```
var ch = newChannel();
```

An unbuffered channel has zero buffer capacity. A send blocks until a receiver is ready, and a receive blocks until a sender is ready. This provides synchronization between coroutines.

Buffered Channels

```
var ch = newBufferedChannel(capacity);
```

A buffered channel has the given capacity (a non-negative integer). Sends do not block until the buffer is full. Receives do not block as long as the buffer has values.

Send and Receive

```
value -> channel;     // send
var v = <- channel;   // receive
```

Send semantics (in priority order):

1. If a receiver is waiting: deliver directly and schedule the receiver.
2. If the buffer is not full: enqueue in buffer.

3. If the buffer is full (or unbuffered): block the sender (max send queue: 4).

Receive semantics (in priority order):

1. If there are pending senders (unbuffered): take the value, schedule the sender.
2. If there are pending senders and buffered values: take from buffer, move sender's value to buffer, schedule sender.
3. If the buffer has values but no pending senders: take from buffer.
4. If empty: block the receiver (max receive queue: 4).

6.3 Scheduling

Coroutines are scheduled using a priority queue keyed by time (the **global scheduler queue**). `yield` re-enqueues at the current time. `sleep` enqueues at `now + duration`. The scheduler always runs the coroutine with the earliest scheduled time.

When a coroutine created via `sleep` is dequeued, the runtime waits (via a readiness signal) until the sleep duration has actually elapsed before resuming it.

Channel Queues

In addition to the global scheduler queue, each channel maintains its own **send queue** and **receive queue** for coroutines blocked on channel operations:

- When a sender blocks (no receiver ready, buffer full), it is added to the channel's send queue.
- When a receiver blocks (no sender ready, buffer empty), it is added to the channel's receive queue.
- These are separate from the global scheduler queue.

The global scheduler queue only contains coroutines that are runnable (not blocked on channels). Coroutines blocked on channel operations remain in the channel's queue until unblocked by the other end.

7. Builtin Functions

The following functions are predefined in the global environment:

Function	Arity	Description
`print(value)`	1	Prints the value's string representation (see §2.3) to stdout followed by a newline, and returns `null`. For strings, the output is the raw string without quotes.
`newChannel()`	0	Creates and returns a new unbuffered channel.

Function	Arity	Description
`newBufferedChannel(cap)`	1	Creates and returns a buffered channel with the given capacity (non-negative integer).
`sleep(ms)`	1	Suspends the current coroutine for `ms` milliseconds (non-negative integer). Returns `null`.
`getCurrentMillis()`	0	Returns the current POSIX time in milliseconds as a number.

Builtins can be reassigned (via `=`) but not redefined (via `var`).

8. Error Handling

All errors in Co are fatal and terminate the program with a non-zero exit code. Error messages are printed to stderr, prefixed with `ERROR:`. There is no try/catch mechanism.

8.1 Run-time Errors

All errors in Co are detected and raised at run-time. The following errors are raised during execution:

Error	Condition
`Unknown variable: <name>`	Reference to an undefined variable.
`Variable already defined: <name>`	Redefinition of a variable in the same or enclosing scope.
`Cannot return from global scope`	`return` executed at global scope (including inside `if`/`while` at global scope).
`Function name same as parameter name: <name>`	A parameter shares the function's name.
`Duplicate paramater names: <names>`	Two or more parameters have the same name.
`Cannot add or append: <l> and <r>`	`+` with incompatible non-numeric, non-string operands.
`Cannot subtract non-numbers: <l> and <r>`	`-` with non-numeric operands.
`Cannot divide non-numbers: <l> and <r>`	`/` with non-numeric operands.
`Cannot multiply non-numbers: <l> and <r>`	`*` with non-numeric operands.
`Cannot compare non-numbers: <l> and <r>`	`<` or `>` with non-numeric operands.

Error	Condition
`Division by zero`	Division by zero
`sleep argument is not a non-negative number: <val>`	`sleep` called with a non-numeric or negative argument.
`newBufferedChannel argument is not a non-negative number: <val>`	`newBufferedChannel` called with a non-numeric or negative argument.
`Cannot call a non-function: <expr> is <val>`	Call expression on a non-function value.
`<name> called with wrong number of arguments: got <m>, expected <n>`	Wrong number of arguments in function call.
`Cannot send to a non-channel: <val>`	Send to a non-channel value.
`Cannot receive from a non-channel: <val>`	Receive from a non-channel value.
`Channel send queue is full`	Send queue exceeds max size (4).
`Channel receive queue is full`	Receive queue exceeds max size (4).
`Deadlock: main coroutine blocked on channel with no runnable coroutines`	Program exits with main coroutine blocked but no runnable coroutines in global queue.

9. Program Structure

A Co program is a sequence of statements executed top-to-bottom. The program file is parsed in its entirety before any execution or analysis begins.

```
program ::= stmt*
```

After the main body finishes, the runtime enters an **await termination** phase:

1. The scheduler runs coroutines from the global scheduler queue (created via `yield`, `sleep`, or `spawn`).
2. Coroutines blocked on channel operations remain in the channel's queue and are **not** added in the global scheduler queue.
3. The program terminates when the global scheduler queue is empty.
4. If the main coroutine is blocked on a channel (in a channel receive/send queue) but the global scheduler queue is empty, the program terminates with a **deadlock error**:

   ```
   ERROR: Deadlock: main coroutine blocked on channel with no runnable coroutines
   ```

This means that channel operations in the main coroutine will only succeed if there are other runnable coroutines (via `spawn`, `yield`, or `sleep`) that can

unblock them. If the main coroutine blocks on a channel and no other coroutines can run, the program will terminate with a deadlock error.

10. Grammar Summary

This grammar is informal; see §3.3 for the actual operator precedence table.

```
program       ::= stmt*

stmt          ::= ifStmt
                | whileStmt
                | varStmt
                | yieldStmt
                | spawnStmt
                | returnStmt
                | functionStmt
                | assignStmt
                | sendStmt
                | exprStmt

ifStmt        ::= "if" "(" expr ")" "{" stmt* "}"
whileStmt     ::= "while" "(" expr ")" "{" stmt* "}"
varStmt       ::= "var" IDENT "=" expr ";"
yieldStmt     ::= "yield" ";"
spawnStmt     ::= "spawn" expr ";"
returnStmt    ::= "return" expr? ";"
functionStmt  ::= "function" IDENT "(" params? ")" "{" stmt* "}"
assignStmt    ::= IDENT "=" expr ";"
sendStmt      ::= expr "->" expr ";"
exprStmt      ::= expr ";"

params        ::= IDENT ("," IDENT)*

(* Expression parsing uses a precedence-climbing algorithm.
   The following is a simplified representation. *)

expr          ::= equalityExpr
equalityExpr  ::= comparisonExpr (("==" | "!=") comparisonExpr)*
comparisonExpr::= addExpr (("<" | ">") addExpr)*
addExpr       ::= mulExpr (("+" | "-") mulExpr)*
mulExpr       ::= unaryExpr (("*" | "/") unaryExpr)*
unaryExpr     ::= "<-" unaryExpr | callExpr
callExpr      ::= primary ("(" args? ")")*    (* left-to-right chaining *)
args          ::= expr ("," expr)*
```

```
primary       ::= "null"
                | "true"
                | "false"
                | STRING
                | INTEGER
                | lambdaExpr
                | IDENT
                | "(" expr ")"

lambdaExpr    ::= "function" "(" params? ")" "{" stmt* "}"

IDENT         ::= LETTER (LETTER | DIGIT)*
INTEGER       ::= ("-")? DIGIT+
STRING        ::= '"' (ESCAPED | UNESCAPED)* '"'
ESCAPED       ::= '\' ('\' | '"' | 'n' | 't' | 'r')
UNESCAPED     ::= char

COMMENT       ::= "//" [^\n]* | "/*" ([^*] | "*" [^/])* "*/"
```

11. Example Programs

This section contains larger example programs demonstrating Co's features.

11.1 Producer-Consumer with Channels

A classic concurrency pattern where producers send data to a shared channel and consumers receive from it:

```
// Producer-Consumer: spawns producers and consumers communicating via channel
var ch = newBufferedChannel(5);
var result = 0;

// Producer: sends numbers 1 through n
function producer(n) {
  var i = 1;
  while (i < n + 1) {
    i -> ch;
    i = i + 1;
  }
}

// Consumer: receives and sums all values
function consumer(n) {
  var sum = 0;
  var count = 0;
  while (count < n) {
```

```
    var v = <- ch;
    sum = sum + v;
    count = count + 1;
  }
  result = sum;
}

spawn producer(5);
consumer(5);
print(result);  // 15
```

11.2 Curried Higher-Order Functions

Functions can return functions, enabling currying and functional patterns:

```
// Curried add: partial application
var add = function(x) {
  return function(y) {
    return x + y;
  };
};

var add5 = add(5);
print(add5(3));  // 8
```

11.3 Linked-Lists and Map

Co has no built-in data structures, but they can be simulated using closures:

```
// Linked-list implementation using closures
function Cons(first, rest) {
  return function (command) {
    if (command == "first") { return first; }
    if (command == "rest") { return rest; }
  };
}

function Empty() { return null; }

function first(list) { return list("first"); }
function rest(list) { return list("rest"); }
function prepend(list, element) { return Cons(element, list); }

// Map: applies a function to each element of a list
function map(fn, list) {
  if (list == null) { return Empty(); }
  return prepend(map(fn, rest(list)), fn(first(list)));
```

```
}

// Build a list from 1 to n
function buildList(n) {
  var result = Empty();
  var i = n;
  while (i > 0) {
    result = prepend(result, i);
    i = i - 1;
  }
  return result;
}

// Print each element of a list
function printList(list) {
  if (list == null) { return; }
  print(first(list));
  printList(rest(list));
}

// Test: double each element in list [3, 2, 1]
var list = buildList(3);
print("Original:");
printList(list);

var doubled = map(function(x) { return x * 2; }, list);
print("Doubled:");
printList(doubled);

// Output:
// Original: 1, 2, 3
// Doubled: 2, 4, 6
```

11.4 Mutual Recursion

Co supports mutual recursion through outer-scope variable binding:

```
// Mutual recursion: even/odd detection
var isEvenFn = null;
var isOddFn = null;

function isEven(n) {
  if (n == 0) { return true; }
  return isOddFn(n - 1);
}
```

```
function isOdd(n) {
  if (n == 0) { return false; }
  return isEvenFn(n - 1);
}

// Bind after declaration
isEvenFn = isEven;
isOddFn = isOdd;

print(isEven(42));  // true
print(isEven(7));   // false
print(isOdd(7));    // true
```

11.5 Fibonacci

A recursive fibonacci implementation:

```
// Fibonacci: naive recursive implementation
function fib(n) {
  if (n == 0) { return 0; }
  if (n == 1) { return 1; }
  return fib(n - 1) + fib(n - 2);
}

print(fib(10));  // 55
```

11.6 Closures and State

Demonstrating how closures capture and mutate outer scope variables:

```
// Counter factory: each call creates an independent counter
function makeCounter() {
  var count = 0;

  function increment() {
    count = count + 1;
    return count;
  }

  function get() {
    return count;
  }

  return function(op) {
    if (op == "inc") { return increment(); }
    if (op == "get") { return get(); }
  };
```

```
}

var counter1 = makeCounter();
var counter2 = makeCounter();

print(counter1("inc"));  // 1
print(counter1("inc"));  // 2
print(counter1("get"));  // 2
print(counter2("inc"));  // 1 (independent counter)
```

11.7 Ping-Pong

Two coroutines exchanging messages via channels:

```
// Ping-pong: two coroutines exchanging messages
var ch1 = newChannel();
var ch2 = newChannel();
var rounds = 5;

function player(name, sendCh, recvCh) {
  while (rounds > 0) {
    var msg = <- recvCh;
    print(name + " received: " + msg);
    rounds = rounds - 1;
    if (rounds > 0) {
      msg -> sendCh;
    }
  }
}

spawn player("A", ch1, ch2);
spawn player("B", ch2, ch1);

// Start the game
"ping" -> ch1;

// Output: A and B alternate receiving ping 5 times each
```

11.8 Barrier Synchronization

Multiple coroutines coordinate using a barrier pattern:

```
// Barrier: wait for all workers to complete
var numWorkers = 3;
var barrier = newChannel();

function worker(id) {
```

```
  print("worker " + id + " starting");
  sleep(10);
  print("worker " + id + " arrived at barrier");
  1 -> barrier;
}

spawn worker(0);
spawn worker(1);
spawn worker(2);

// Wait for all workers to arrive
var i = 0;
while (i < numWorkers) {
  <- barrier;
  i = i + 1;
}

print("all workers arrived");
```

11.9 Timing and Scheduling

Using `sleep` and `yield` for cooperative multitasking:

```
// Timed execution: demonstrates sleep and scheduling
var start = getCurrentMillis();

function delayed(name, ms) {
  sleep(ms);
  return name;
}

// Spawn with different delays - order depends on sleep duration
spawn function() { print(delayed("first", 100)); }();
spawn function() { print(delayed("second", 50)); }();
spawn function() { print(delayed("third", 20)); }();

sleep(150);  // Wait for all to complete
var elapsed = getCurrentMillis() - start;
print("completed in " + elapsed + "ms");
```